

Guida all'integrazione con l'IDP

NewTimeGroup

1. Obiettivo

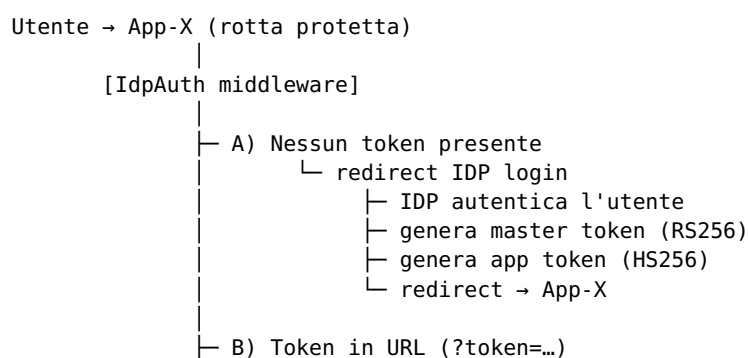
Ottenere **username e ruoli** dell'utente autenticato per gestire l'accesso ai componenti dell'applicazione. Il sistema IDP centralizza l'autenticazione e distribuisce i ruoli specifici di ogni applicazione tramite token JWT.

2. Concetti fondamentali

Entità	Descrizione
Provider	L'applicazione esterna registrata sull'IDP. Riceve un ID univoco e una <code>secret_key</code> a 32 caratteri.
Ruolo	Permesso specifico per un Provider (es. admin, editor). I ruoli di un Provider non hanno effetto sugli altri.
ProviderUserRole	Relazione che assegna un Ruolo a un Utente per un Provider specifico.
Master Token	Token JWT RS256 emesso al login dall'IDP centrale. Contiene i dati anagrafici dell'utente. Condiviso tra tutte le app SSO.
App Token	Token JWT HS256 specifico per ogni Provider. Contiene <code>user.id</code> e i ruoli dell'utente per quella specifica applicazione.

3. Schema del flusso SSO

Flusso web (browser)



App Token (HS256)

Firmato con la `secret_key` del Provider (`IDP_SIGNATURE_KEY`). Contiene i ruoli specifici di quella applicazione.

Esempio decodificato:

```
{
  "iss": "https://idp-staging.newtimegroup.it",
  "iat": 1780912245,
  "exp": 1780914045,
  "nbf": 1780912245,
  "jti": "bc89e1b209d1ad65f538",
  "sub": "8",
  "prv": 2,
  "aud": "app-x.example.com",
  "payload": {
    "user": {
      "id": 8
    },
    "roles": [{ "id": 3, "name": "admin" }]
  }
}
```

Campo	Descrizione
<code>prv</code>	ID del Provider (corrisponde a <code>IDP_CLIENT_ID</code>)
<code>aud</code>	Dominio del Provider
<code>payload.user.id</code>	ID dell'utente sull'IDP
<code>payload.roles</code>	Array di ruoli assegnati all'utente per questo Provider

Nota: `payload.user` nell'app token contiene solo `id`. Per ottenere nome, email e username occorre decodificare il master token (`idp-master-token`).

5. Cookie e trasporto del token

Ambiente	Meccanismo	Nome cookie
Produzione	Cookie HTTP-Only, SameSite=Lax, Secure se HTTPS	<code>idp_token_{IDP_CLIENT_ID}</code>
	Master token accodato come ? token=... nella URL di	
Localhost/sviluppo	redirect; idp- extension lo converte automaticamente in cookie	<code>idp_token_{IDP_CLIENT_ID}</code>

Il nome del cookie è `idp_token_` seguito dall'ID del Provider (`IDP_CLIENT_ID`). Esempio: se il Provider ha ID 5, il cookie si chiama `idp_token_5`.

6. Integrazione con idp-extension

Il pacchetto **idp-extension** gestisce internamente l'intero ciclo SSO (redirect, token exchange, cookie). È disponibile per:

- **PHP (Laravel)** — pacchetto Composer
- **Node.js** — pacchetto npm

Per altri linguaggi o framework, contattare il team IDP per valutare la disponibilità di un'estensione o di una libreria dedicata.

6.1 Configurazione .env (Laravel e Node.js)

```
# Obbligatorie
IDP_CLIENT_ID=<ID del Provider assegnato dall'IDP>
IDP_URL_WEB="https://idp-staging.newtimegroup.it"
IDP_SIGNATURE_KEY=<stringa 32 caratteri alfanumerici, condivisa con
il team IDP>
IDP_URL_TOKEN_EXCHANGE=api/v1/token/exchange

# Opzionali
IDP_URL_JWKS=.well-known/jwks.json
IDP_REQUEST_TIMEOUT_SEC=5
# Solo se app e IDP girano in Docker su reti isolate:
IDP_URL_M2M="http://host.docker.internal:8001"
```

IDP_CLIENT_ID: è l'ID del record Provider creato sull'IDP. Il team IDP lo comunica al momento della registrazione.

IDP_SIGNATURE_KEY: è la secret_key configurata sul Provider nell'IDP, usata per firmare e verificare l'app token. Va concordata tra i due team prima dell'integrazione e comunicata fuori banda (non è esposta dalle API).

6.2 Utilizzo del middleware (Laravel)

```
// routes/web.php
Route::middleware(["idp.auth"])->group(function () {
    Route::get("/dashboard", [DashboardController::class, "index"]);
});
```

6.3 Leggere username e ruoli

Dal token già verificato dal middleware è possibile accedere a:

```
// Username e dati anagrafici → dal master token (cookie idp-master-token)
$user = $request->idp_user; // { id, username, email, name, surname }

// Ruoli → dall'app token (cookie idp token {IDP_CLIENT_ID})
$roles = $request->idp_roles; // [{ id, name }, ...]

// Verificare un ruolo specifico
$isAdmin = collect($roles)->contains("name", "admin");
```

Verificare la documentazione specifica di idp-extension per i nomi esatti degli attributi esposti dal middleware.

7. Procedura onboarding Provider — operazioni lato IDP

Queste operazioni vengono eseguite dal team IDP prima dell'integrazione.

Step 1 — Creare il client Passport

```
php artisan passport:client --password --name="nome-client-esterno"
```

Il nome non deve corrispondere a un username presente nella tabella users. Il comando restituisce client_id e client_secret da fornire al team esterno per l'accesso allo Swagger.

Step 2 — Accedere a Swagger

URL: <https://idp-staging.newtimegroup.it/api/documentation>

Step 3 — Autenticarsi in Swagger

1. Cliccare su **Authorize**
2. Nella sezione OAuth2, inserire client_id e client_secret ottenuti al passo precedente
3. Cliccare **Authorize** per ottenere il Bearer token

Step 4 — Creare il Provider

POST /api/v1/providers

```
{
  "name": "Nome applicazione esterna",
  "url": "https://app-x.example.com",
  "domain": "app-x.example.com",
  "protocol": "https",
  "logoutUrl": "https://app-x.example.com/logout",
  "secret_key": "<stringa 32 caratteri alfanumerici>"
}
```

Risposta: { "provider": { "id": 5, ... } } — annotare l'id, è il valore di IDP_CLIENT_ID.

La secret_key deve essere comunicata al team esterno come IDP_SIGNATURE_KEY **fuori banda** (non è esposta nelle risposte API).

Step 5 — Creare i ruoli del Provider

POST /api/v1/roles

```
{
  "name": "admin",
  "provider_id": 5
}
```

Ripetere per ogni ruolo necessario (es. editor, viewer).

Step 6 — Assegnare i ruoli agli utenti

POST /api/v1/provider-user-roles

```
{
  "provider_id": 5,
  "user_id": 8,
  "role_id": 3
}
```

Ripetere per ogni coppia utente-ruolo.

Step 7 — Verifica

Effettuare il login con un utente assegnato e verificare che il cookie `idp_token_5` sia presente e contenga i ruoli attesi nel payload.

8. Endpoint di riferimento per l'integrazione

Questi endpoint sono usati direttamente da `idp-extension` e raramente vanno invocati manualmente. Sono documentati qui per completezza e per chi integra su stack non supportati.

Login web

POST /v2/login

Content-Type: application/x-www-form-urlencoded

username=mario.rossi&password=secret&provider_id=5

Redirect al Provider con i cookie impostati (o token in URL su localhost).

Token exchange (master → app token)

POST /api/v1/token/exchange

Authorization: Bearer <master_token>

Content-Type: application/json

```
{
  "provider_id": 5,
  "ip_address": "1.2.3.4",
  "user_agent": "Mozilla/5.0 ..."
}
```

Risposta:

```
{ "token": "eyJhbGc..." }
```

Verifica sessione (M2M)

GET /api/v1/sessions/check
Authorization: Bearer <app_token>

Risposta:

```
{ "valid": true, "message": "...", "token": null }
```

9. Stack non supportati da idp-extension

Se l'applicazione non è sviluppata in PHP (Laravel) o Node.js, il team di sviluppo NewTimeGroup deve essere contattato per gestire l'autenticazione.